

1

Form Validation

What is form validation?

2

- **validation:** ensuring that form's values are correct
- some types of validation:
 - preventing blank values (email address)
 - ensuring the type of values
 - integer, real number, currency, phone number, Social Security number, postal
 - address, email address, date, credit card number, ...
 - ensuring the format and range of values (ZIP code must be a 5-digit integer)
 - ensuring that values fit together (user types

A real Form that uses validation

3

The screenshot shows a Firefox browser window with multiple tabs. The active tab is 'WeightWatchers.com: Si...'. The address bar shows the URL 'https://signup.weightwatchers.com/SignupVersions/registration/StepOne.aspx'. The page title is 'sign up weightwatchers index'. The page content includes a sidebar on the left with the heading 'Become a Registered User!' and links to 'Start a blog', 'Participate in a Challenge', 'Save your favorite recipes', and 'Post to our boards'. The main content area is titled 'Create Your Registered User Account Login' and contains a registration form. The form has several fields with validation errors indicated by red 'X' marks and error messages:

- First name:** Xenia (no error)
- Last name:** (empty) **X** Please enter your last name. It's required information.
- Your birthdate:** Month (dropdown), Day (dropdown), Year (dropdown) **X** Please select your month, day, and year of birth. It's required information.
- Your gender:** ☐ Female ☐ Male **X** Please enter your gender. It's required information.
- State:** (Select One) (dropdown) **X** Please choose a state. It's required information.
- Zip code:** (empty) **X** Please enter a 5-digit ZIP code. It's required information.
- E-mail:** (empty) **X** Please enter your email address in the following format: abc@example.com. It's required information.
- Re-enter e-mail:** (empty)

The browser's status bar at the bottom shows 'Find: prereq' and navigation buttons for 'Next', 'Previous', 'Highlight all', and 'Match case'.

Client vs. server-side validation

4

- Validation can be performed:
 - **client-side** (before the form is submitted)
 - can lead to a better user experience, but not secure (why not?)
 - **server-side** (in PHP code, after the form is submitted)
 - needed for truly secure validation, but slower
 - best mix of convenience and security, but requires most effort to program

An example form to be validated

5

```
<form action="http://foo.com/foo.php" method="get">
  <div>
    City: <input name="city"> <br>
    State: <input name="state" size="2"
maxlength="2"> <br>
    ZIP: <input name="zip" size="5" maxlength="5">
    <br>
    <input type="submit">
  </div>
</form>
```

HTML

- Let's validate this form's data on the server...

Basic server-side validation code

6

```
<?php
    $city = $_REQUEST["city"];
    $state = $_REQUEST["state"];
    $zip = $_REQUEST["zip"];
    if (!$city || strlen($state) != 2 || strlen($zip) !=
5) {
        ?>
        <h2>Error, invalid city/state submitted.</h2>
        <?php
    }
    ?>
```

PHP

- basic idea: examine parameter values, and if they are bad, show an error message and abort

Basic server-side validation code

7

- validation code can take a lot of time / lines to write
 - How do you test for integers vs. real numbers vs. strings?
 - How do you test for a valid credit card number?
 - How do you test that a person's name has a middle initial?
 - How do you test whether a given string matches a particular complex format?

Regular expressions

8

<code>[a-z]at</code>	<code>#cat, rat, bat...</code>
<code>[aeiou]</code>	
<code>[a-zA-Z]</code>	
<code>[^a-z]</code>	<code>#not a-z</code>
<code>[:alnum:]+</code>	<code>#at least one alphanumeric char</code>
<code>(very) *large</code>	<code>#large, very very very large...</code>
<code>(very){1, 3}</code>	<code>#counting "very" up to 3</code>
<code>^bob</code>	<code>#bob at the beginning</code>
<code>com\$</code>	<code>#com at the end</code>

PHPRegExp

- Regular expression: a pattern in a piece of text
- PHP has:
 - POSIX
 - **Perl regular expressions**

Delimiters

9

<code>/[a-z]/at</code>	<code>#cat, rat, bat...</code>
<code>#[aeiou]#</code>	
<code>/[a-zA-Z]/</code>	
<code>~[^a-z]~</code>	<code>#not a-z</code>
<code>/[[:alnum:]]+/</code>	<code>#at least one alphanumeric char</code>
<code>*(very) *#large</code>	<code>#large, very very very large...</code>
<code>~(very){1, 3}~</code>	<code>#counting "very" up to 3</code>
<code>/^bob/</code>	<code>#bob at the beginning</code>
<code>/com\$/</code>	<code>#com at the end</code>
<code>/http:\\/\\</code>	
<code>// #http://#</code>	<code>#better readability</code>

PHPRegExp

- Used for Perl regular expressions (preg)

Basic Regular Expression

10

```
/abc/
```

- in PHP, regexes are strings that begin and end with /
- the simplest regexes simply match a particular substring
- the above regular expression matches any string containing "abc":
 - YES: "abc", "abcdef", "defabc", " .=.abc.=.", ...
 - NO: "fedcba", "ab c", "PHP", ...

Wildcards

11

- A dot `.` matches any character except a `\n` line break
 - `"/.oo.y/"` matches "Doocy", "goofy", "LooNy", ...
- A trailing `i` at the end of a regex (after the closing `/`) signifies a case-insensitive match
 - `"/xen/i"` matches "Xenia", "xenophobic", "Xena the warrior princess", "XEN technologies" ...

Special characters: |, (), ^, \

12

- | means *OR*
 - `"/abc|def|g/"` matches `"abc"`, `"def"`, or `"g"`
 - There's no *AND* symbol. Why not?
- () are for grouping
 - `"/(Homer|Marge) Simpson/"` matches `"Homer Simpson"` or `"Marge Simpson"`
- ^ matches the beginning of a line; \$ the end
 - `"/^<!--$/"` matches a line that consists entirely of `"<!--"`

Special characters: |, (), ^, \

13

- \ starts an escape sequence
 - many characters must be escaped to match them literally: / \ \$. [] () ^ * + ?
 - `"/<br \>/"` matches lines containing `<br />` tags

Quantifiers: *, +, ?

14

- * means 0 or more occurrences
 - `/abc*/` matches "ab", "abc", "abcc", "abccc", ...
 - `/a(bc)*/` matches "a", "abc", "abcbc", "abcbcbc", ...
 - `/a.*a/` matches "aa", "aba", "a8qa", "a!?!_a", ...
- + means 1 or more occurrences
 - `/a(bc)+/` matches "abc", "abcbc", "abcbcbc", ...
 - `/Goo+gle/` matches "Google", "Gooogle", "Goooooogle", ...
- ? means 0 or 1 occurrences
 - `/a(bc)?/` matches "a" or "abc"

15

- CS380

Character sets: []

16

- [] group characters into a character set; will match any single character from the set
 - `"/[bcd]art/"` matches strings containing "bart", "cart", and "dart"
 - equivalent to `"/(b|c|d)art/"` but shorter
- inside [], many of the modifier keys act as normal characters
 - `"/what[!*?]*/"` matches "what", "what!", "what? **!", "what??!",
- What regular expression matches DNA (strings of A, C, G, or T)?

Character ranges: [start-end]

17

- inside a character set, specify a range of characters with -
 - `"/[a-z]/"` matches any lowercase letter
 - `"/[a-zA-Z0-9]/"` matches any lower- or uppercase letter or digit
- an initial `^` inside a character set negates it
 - `"/[^abcd]/"` matches any character other than a, b, c, or d

Character ranges: [start-end]

18

- inside a character set, - must be escaped to be matched
 - `"/[+\-]?[0-9]+/"` matches an optional + or -, followed by at least one digit
- What regular expression matches letter grades such as A, B+, or D- ?

Escape sequences

19

- special escape sequence character sets:
 - **\d** matches any digit (same as [0-9]); **\D** any non-digit ([^0-9])
 - **\w** matches any “word character” (same as [a-zA-Z_0-9]); **\W** any non-word
- char
 - **\s** matches any whitespace character (, \t, \n, etc.); **\S** any non-whitespace
- What regular expression matches dollar amounts of at least \$100.00 ?

Regular expressions in PHP

(PDF)

20

- regex syntax: strings that begin and end with /, such as `"/[AEIOU]+/"`

function	description
<code><u>preg_match</u>(regex, string)</code>	returns TRUE if string matches regex
<code><u>preg_replace</u>(regex, replacement, string)</code>	returns a new string with all substrings that match regex replaced by replacement
	returns an array of strings from given string broken apart using the given

Regular expressions example

21

```
echo preg_match ('/test/', "a test of preg_match");  
echo preg_match ('/  
tutorial/', "a test of preg_match");  
  
$matchesarray[0] = "http://www.tipsntutorials.com/"  
$matchesarray[1] = "http://"  
$matchesarray[2] = "www.tipsntutorials.com/"  
echo  
preg_match ('/(http:\\/\\/)(.*)/', "http://www.tipsntu  
torials.com/", $matchesarray);
```

PHP

Regular expressions example

22

```
# replace vowels with stars
$str = "the quick brown fox";
$str = preg_replace("/[aeiou]/", "*", $str);
# "th* q**ck br*wn f*x"
# break apart into words
$words = preg_split("/[ ]+/", $str);
# ("th*", "q**ck", "br*wn", "f*x")
# capitalize words that had 2+ consecutive vowels
for ($i = 0; $i < count($words); $i++) {
    if (preg_match("/\\{2,}/", $words[$i])) {
        $words[$i] = strtoupper($words[$i]);
    }
} # ("th*", "Q**CK", "br*wn", "f*x")
```

PHP

PHP form validation w/ regexes

23

```
$state = $_REQUEST["state"];  
if (!preg_match("/[A-Z]{2}/", $state)) {  
?>  
<h2>Error, invalid state submitted.</h2>  
<?php  
}
```

PHP

- using preg_match and well-chosen regexes allows you to quickly validate query parameters against complex patterns

Another PHP experiment

24

- Write a PHP script that tests whether an e-mail address is input correctly. Test using valid and invalid addresses
- Use array
- Use function